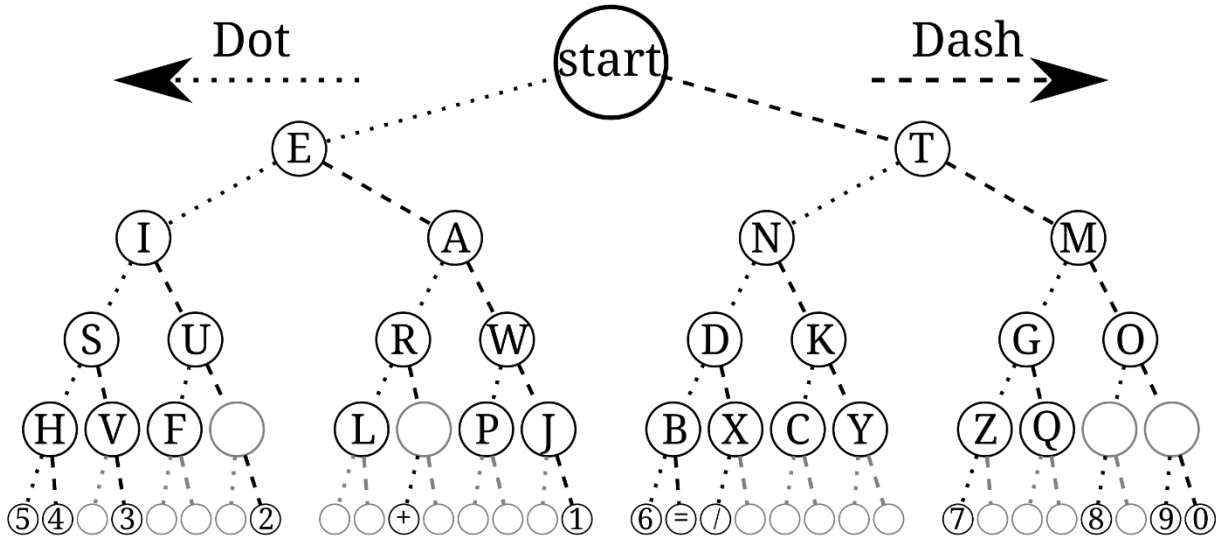

Developer Documentation

Morse code

—

Noor Almukhtar

Introduction:

- The program deals with the encoding and decoding of Morse code. It takes data from the user and transforms it into Morse code, and Morse code into text.
- A command line can be used to encode/decode and their statistics too.
- The data of the morse or text will be saved in a file for the third option in the menu, character statistics.
- Character statistics indicates how many numbers/ letters/ dots/ dashes we have, and it counts it.

Details of the program:

1. Included header files:

- #include <stdio.h>
- #include <stdlib.h>
- #include <string.h>
- #include <ctype.h>

2. File handling implementation:

- FILE *fp; //file pointer so fp is created
- fprintf(fp, " ",); //printing the result in txt file to store it
- fp = fopen("statistics.txt", "a"); //file pointer to open the file to append it
- fclose(fp); //file pointer that closes the file

3. Data structures used in the code:

1) Node Structure:

- A structure named tree_node represents a node in a binary tree.
- Each node contains the following components:
  - char data: Represents the character associated with the node.
  - struct tree_node *left: Pointer to the left child node.
  - struct tree_node *right: Pointer to the right child node.

2) Node Creation Function:

- create_node(char data): This function creates a new node, initializes its data with the provided character (data), and sets left and right child pointers to NULL.
- The function returns a pointer to the newly created node.

3) Notes:

- The `tree_node` structure is a fundamental building block for constructing binary trees.
- The `create_node` function simplifies the process of creating new nodes by handling memory allocation and initialization.
- The function ensures that the left and right child pointers are initialized to `NULL`, indicating that the new node is initially a leaf node with no children.
- The `malloc` function is used for dynamic memory allocation, and it is essential to free the allocated memory using `free` when nodes are no longer needed to prevent memory leaks.
- This structure and function are part of the Morse code encoder/decoder program, where binary trees are employed for decoding Morse code.

```c
//it is like a container that holds a letter or number and two pointers to make a tree.
typedef struct tree_node {

    char data; //the letter or number that this tree node has.

    struct tree_node *left;      //points to the left branch in the tree.
    struct tree_node *right;     //points to the right branch in the tree.

} tree_node;



//this function makes a new tree node and puts a letter or number inside it.
tree_node *create_node(char data) {

 //asking the computer for some memory to hold our node.
    tree_node* new_node = (tree_node*)malloc(sizeof(tree_node));

    if(new_node != NULL) { //if we got the memory:

        new_node->left = NULL;        //setting the left branch to nothing.
        new_node->right = NULL;      //setting the right branch to nothing.
        new_node->data = data;      //puting the data (letter or number) inside the node.
    }
    return new_node; //gives us back the new node.
}
```

4) main(...):

- **Description:**
  - **The program can run in two modes:**
    - **command-line mode or the interactive menu.**
  - **Command-line mode (when arguments are given):**
    - **morse.exe 1 <text> → Encodes text.**
    - **morse.exe 2 <morse_code> → Decodes Morse.**
    - **All words after the mode flag are joined into one input string.**
    - **The program prints the result and statistics, then exits.**
  - **Menu mode (when no arguments are provided):**
    - **Encode text → Morse.**
    - **Decode Morse → text.**
    - **View/reset character statistics.**
    - **Exit the program.**
  - **The program uses a binary tree structure for decoding Morse code, and it logs the user's input and program output to a file named "statistics.txt".**

5) Program Structure:

- **Node Structure:**
  - **node: Represents a node in the binary tree structure.**
  - **Each node contains a character and pointers to its left and right children.**

- **Binary Tree Building Function:**
  - **build_tree: Constructs a binary tree based on Morse code representations. This function is used to decode Morse code.**

- **Memory Deallocation Function:**

- delete: Recursively deletes all nodes in a binary tree, preventing memory leaks.

- **Text to Morse Code Encoding Function:**
  - encode_morse: Takes user input text and encodes it into Morse code using a predefined mapping.

- **Morse Code to Text Decoding Function:**
  - decode_morse: Takes user input Morse code and decodes it into text using the binary tree structure built with build_tree.

- **User Interface and Menu:**
  - In the main function, the program displays a menu with the following options:
    - Text to Morse Code
    - Morse Code to Text
    - Character statistics
    - Exit

- **File Handling:**
  - Details about the character statistics and logs in "Statistics.txt"

- **Main Program Loop:**
  - The program runs an infinite loop where the user can select menu items:
    - Option 1: the user can input text for encoding.
    - Option 2: the user can input Morse code for decoding.
    - Option 4: displays the program char. Statistics and reset.
    - Option 3: exits the program.

- **Memory Cleanup and Program Exit:**

- The program closes the file pointer, frees memory associated with the binary tree, and exits when the user chooses the exit option.

- **Usage:**
  - Compile and run the program.
  - Select menu options to perform text-to-Morse or Morse-to-text conversions, view char. statistics, or exit the program.

Functions used in the program:

I. tree_node *create_node(...) – makes a new tree putting the info in
II. char *input () – takes an undefined length of data
III. encode_morse(...) – text to morse
IV. decode_morse(...) – morse to text
V. build_tree1(...) – create a binary tree to store Morse code
VI. delete(...) – deletes the tree to free memory
VII. char_statistics(...) – shows the char. statistics from the input
VIII. reset_statistics(...) – reset char. statistics back to 0
IX. main(...) – has the menu options

Explanation of functions:

1) tree_node *create_node(...)

- **Description:**
  - This function creates a new node for the Morse code binary tree. The node stores the given character (data) and initializes pointers to its left and right child nodes as NULL.
- **Parameters:**

- char data: The character to be stored in the node (e.g., a letter, number, or special character).

- **Return type:**
  - Returns a pointer to the newly created tree_node.
- **Notes:**
  - Dynamically allocates memory using malloc.
  - If memory allocation fails, the function returns NULL.
  - This function is a fundamental building block for constructing the binary tree used in decoding Morse code.
- **Code:**

```
24  //this function makes a new tree node and puts a letter or number inside it.
25  tree_node *create_node(char data) {
26
27   //asking the computer for some memory to hold our node.
28      tree_node* new_node = (tree_node*)malloc(sizeof(tree_node));
29
30      if(new_node != NULL) { //if we got the memory:
31
32          new_node->left = NULL;        //setting the left branch to nothing.
33          new_node->right = NULL;      //setting the right branch to nothing.
34          new_node->data = data;      //puting the data (letter or number) inside the node.
35      }
36      return new_node; //gives us back the new node.
37  }
```

2) char *input()

- **Description:**
  - Reads input from the user character by character, dynamically allocating memory to store the input string. The function resizes the allocated memory as more characters are read.

- **Parameters:**
  - None.

- **Return type:**
  - Returns a pointer to a dynamically allocated character array containing the user's input.

- **Notes:**
  - Uses `malloc` for the initial allocation and `realloc` to expand the memory as needed.
  - Ensures the input string is null-terminated (`\0`).
  - Stops reading when a newline character (`\n`) is encountered.
  - If memory allocation fails at any stage, it prints an error message and terminates the program.

- **Code:**

```c
//function to read input dynamically, character by character
char *input() {

    int cnt = 0;    //counter to keep track of the number of characters read

    char *line = (char *) malloc(sizeof(char) * 1);     //allocating initial memory for line

    if (line == NULL) {     //check if memory allocation failed
        fprintf(stderr, "Error: Unable to allocate memory\n");  //(standard error, print error message)
        exit(1);     //exit the program due to memory allocation failure
    }

    line[0] = '\0';  //set the initial line to be an empty string

    char newchar;   //variable to hold each character read

    while (scanf("%c", &newchar) == 1 && newchar != '\n') {     //reads until newline character is found

        cnt++; //increment character count

        char *newarr = realloc(line, sizeof(char) * (cnt + 1));     //reallocate memory for new character and null terminator (+1)

        if (newarr == NULL) {       //check if memory allocation failed
            fprintf(stderr, "Error: Unable to allocate memory\n");      //(standard error, print error message)
            free(line);         //free previously allocated memory to avoid leaks
            exit(1);        //exit program
        }

        line = newarr;      //point line to the new memory
        line[cnt - 1] = newchar;        //add the new character to the array
        line[cnt] = '\0';       //null-terminate the string
    }

    return line;    //return the dynamically allocated input string
}
```

3) void encode_morse(...)

- **Description:**
  - Converts the input string into its Morse code representation, updating statistics for the number of letters, numbers, dots, and dashes used in the encoding.
- **Parameters:**
  - `char *input`: The string to be encoded into Morse code.
  - `int *letters`: Pointer to the count of letters in the input.
  - `int *dots`: Pointer to the count of dots (.) in the encoded output.
  - `int *dashes`: Pointer to the count of dashes (-) in the encoded output.
  - `int *numbers`: Pointer to the count of numbers in the input.
- **Return type:**
  - None.
- **Notes:**
  - Letters (`a-z`) and numbers (`0-9`) are mapped to their Morse code equivalents using an array.
  - Spaces in the input are handled by inserting double spaces in the Morse output to separate words.
  - Invalid characters are ignored, and a warning is printed.
  - Updates cumulative statistics for letters, numbers, dots, and dashes as it processes the input.

Code:

```c
//function to encode text into Morse code
void encode_morse(char *input, int *letters, int *dots, int *dashes, int *numbers) {

    //writing the characters in morse code:
    //in morse code characters from A to Z then from 0 to 9

    char *morse[] = {
        ".-", "-...", "-.-.", "-..", ".", "..-.", "--.",  // A-G
        "....", "..", ".---", "-.-", ".-..", "--", "-.",  // H-N
        "---", ".--.", "--.-", ".-.", "...", "-", "..-",  // O-U
        "...-", ".--", "-..-", "-.--", "--..",            // V-Z

        "-----", ".----", "..---", "...--", "....-",      // 0-4
        ".....", "-....", "--...", "---..", "----."       // 5-9
    };

    for(int i = 0; i < strlen(input); i++) {

        char c = tolower(input[i]);         //convert character to lowercase for uniform indexing

        if (c >= 'a' && c <= 'z') {         //if character is a letter

            (*letters)++;                   //update letter count

            printf("%s ", morse[c - 'a']);  //print corresponding Morse code

            //update cumulative dot and dash statistics
            for(int j = 0; j < strlen(morse[c - 'a']); j++) {

                if(morse[c - 'a'][j] == '.') {
                    (*dots)++;

                } else if(morse[c - 'a'][j] == '-') {
                    (*dashes)++;
                }
            }

        } else if (c >= '0' && c <= '9') {      //if character is a digit

            (*numbers)++;  //updating number count.

            printf("%s ", morse[c - '0' + 26]);      //print corresponding Morse code

            //update cumulative dot and dash statistics
            for(int j = 0; j < strlen(morse[c - '0' + 26]); j++) {

                if(morse[c - '0' + 26][j] == '.') {
                    (*dots)++;

                } else if(morse[c - '0' + 26][j] == '-') {
                    (*dashes)++;
                }
            }

        } else if (c == ' ') {          //if character is a space

            printf("  ");           //print double spaces to indicate separation between words

        } else {            //handle any invalid character

            printf("Invalid character");
        }
    }
    printf("\n"); //print a newline at the end of the encoded message
}
```

4) void decode_morse(...)

- **Description:**
  - Decodes a Morse code string into plain text using the binary tree and updates character statistics.

- **Parameters:**
  - `tree_node *root`: The root node of the binary tree used for decoding.
  - `char *input`: The Morse code string to be decoded.
  - `int *letters`: Pointer to the count of letters in the decoded text.
  - `int *dots`: Pointer to the count of dots (.) in the input.
  - `int *dashes`: Pointer to the count of dashes (-) in the input.
  - `int *numbers`: Pointer to the count of numbers in the decoded text.

- **Return type:**
  - None.

- **Notes:**
  - Traverses the binary tree based on the dot-dash sequence:
  - A dot (.) moves to the left child.
  - A dash (-) moves to the right child.
  - Spaces indicate the end of a character, and double spaces indicate the end of a word.
  - Resets to the root node after decoding each character.
  - Handles invalid Morse code sequences by printing an error and terminating the decoding process.

- **Code:**

```c
//function to decode Morse code to text (not yet implemented)
void decode_morse(tree_node *root, char *input, int *letters, int *dots, int *dashes, int *numbers) {
    int i = 0;  //starting at the first character

//create a space to store the decoded message
    char *decoded = (char *)malloc(sizeof(char) * (strlen(input) + 1));  // +1 for ending

    if (decoded == NULL) {

        printf("Memory allocation error.\n");
        return;
    }

    int j = 0;  //where we are in the decoded message
    tree_node *temp = root;  //starting at the top of our Morse tree

//goes through each character in the input
    while (i <= strlen(input)) {

        if (input[i] == '.' || input[i] == '-') {

            //traverse through the tree, based on dots and dashes
            if (input[i] == '.') {

                (*dots)++;  //count the dot

                if (root->left != NULL) {

                    root = root->left;  //move left for dot

                } else {

                    printf("\nInvalid Morse code detected.\n");
                    free(decoded);
                    return;
                }

            } else if (input[i] == '-') {

                (*dashes)++;  //count the dash

                if (root->right != NULL) {

                    root = root->right;  //move right for dash

                } else {

                    printf("\nInvalid Morse code detected.\n");
                    free(decoded);
                    return;
                }
            }
```

```
289
290         } else if (input[i] == ' ' || input[i] == '\0') {
291
292             //when we hit a space or the end of input, finish the letter
293             if (root != temp) {
294
295                 decoded[j++] = root->data;  //add the letter to our message
296
297                 //count the letter or number accordingly
298                 if (root->data >= '0' && root->data <= '9') {
299
300                     (*numbers)++;
301
302                 } else if (root->data >= 'a' && root->data <= 'z') {
303
304                     (*letters)++;
305                 }
306
307                 root = temp;  //goes back to the top of the tree
308             }
309
310             //skip multiple spaces between words
311             while (input[i] == ' ' && input[i + 1] == ' ') {
312
313                 i++;
314             }
315
316         } else {
317
318             //if we find a character that doesn't belong
319             printf("\nInvalid Morse code character detected.\n");
320             free(decoded);
321             return;
322         }
323
324         i++;  //move to the next character
325     }
326
327     //handle the last character if there is no space at the end
328     if (root != temp) {
329
330         decoded[j++] = root->data;
331
332         if (root->data >= '0' && root->data <= '9') {
333
334             (*numbers)++;
335
336         } else if (root->data >= 'a' && root->data <= 'z') {
337
338             (*letters)++;
339         }
340     }
341
342     decoded[j] = '\0';  //end the message
343
344     //print the decoded message
345     printf("The text is: %s\n", decoded);
346
347     //frees the space we used to store the message
348     free(decoded);
349 }
350
```

5) void build_tree(...)

- **Description:**
  - Constructs a binary tree to represent Morse code by associating each character with its corresponding dot-dash pattern.
- **Parameters:**
  - tree_node *root: Pointer to the root node of the binary tree.
- **Return type:**
  - None.
- **Notes:**
  - Each level of the tree corresponds to a step in the dot-dash sequence:
  - A dot (.) moves to the left child.
  - A dash (-) moves to the right child.
  - Characters are added at specific nodes based on their Morse code representation.
  - Supports letters (a-z), numbers (0-9), and space characters for decoding.
- **Code:**

```
353 //function to build the morse code tree using binary search trees
354 void build_tree(tree_node* root){
355
356 //creating a binary search tree to store the morse code characte
357
358 //level 1 is the node of the tree (the start)
359
360 //level 2 of the tree
361
362     root->left = create_node('e');  // .
363     root->right = create_node('t'); // -
364
365 //level 3
366
367     root->left->left = create_node('i'); // ..
368     root->left->right = create_node('a'); // .-
369
370     root->right->left = create_node('n'); // -.
371     root->right->right = create_node('m'); // --
372
373 //level 4
374
375     root->left->left->left = create_node('s'); // ...
376     root->left->left->right = create_node('u'); // ..-
377
378     root->left->right->left = create_node('r'); // .-.
379     root->left->right->right = create_node('w'); // .--
380
381     root->right->left->left = create_node('d'); // -..
382     root->right->left->right = create_node('k'); // -.-
383
384     root->right->right->left = create_node('g'); // --.
385     root->right->right->right = create_node('o'); // ---
```

```
386
387  //level 5
388
389      root->left->left->left->left = create_node('h'); // ....
390      root->left->left->left->right = create_node('v'); // ...-
391
392      root->left->left->right->left = create_node('f'); // ..-.
393      root->left->left->right->right = create_node(' '); // ..--
394
395      root->left->right->left->left = create_node('l'); // .-..
396
397      root->left->right->right->left = create_node('p'); // .--.
398      root->left->right->right->right = create_node('j'); // .---
399
400      root->right->left->left->left = create_node('b'); // -...
401      root->right->left->left->right = create_node('x'); // -..-
402
403      root->right->left->right->left = create_node('c'); // -.-.
404      root->right->left->right->right = create_node('y'); // -.--
405
406      root->right->right->left->left = create_node('z'); // --..
407      root->right->right->left->right = create_node('q'); // --.-
408
409      root->right->right->right->left = create_node(' '); // ---.
410      root->right->right->right->right = create_node(' '); // ----
411
412  //level 6
413
414      root->left->left->left->left->left = create_node('5'); // .....
415      root->left->left->left->left->right = create_node('4'); // ....-
416
417      root->left->left->left->right->right = create_node('3'); // ...--
418
419      root->left->left->right->right->right = create_node('2'); // ..---
420
421      root->left->right->right->right->right = create_node('1'); // .----
422
423      root->right->left->left->left->left = create_node('6'); // -....
424
425      root->right->right->left->left->left = create_node('7'); // --...
426
427      root->right->right->right->left->left = create_node('8'); // ---..
428
429      root->right->right->right->right->left = create_node('9'); // ----.
430      root->right->right->right->right->right = create_node('0'); // -----
431
432  //end of the tree
433  }
```

6) void delete_tree(...)

- **Description:**
  - Frees all the memory allocated for the Morse code binary tree by deleting its nodes recursively.
- **Parameters:**
  - tree_node *root: Pointer to the root node of the tree.
- **Return type:**
  - None.
- **Notes:**
  - Recursively deletes nodes in post-order traversal:
    - ❖ Deletes the left subtree.
    - ❖ Deletes the right subtree.
    - ❖ Frees the current node.
  - Prevents memory leaks by ensuring all dynamically allocated memory is released.
- **Code:**

```
437  //function to delete/free the morse code tree
438  void delete_tree(tree_node* root){
439
440  //deleting the binary search tree and freeing the memory allocated to it
441
442      if (root == NULL){
443
444          return;     //if there's nothing to delete, stop
445      }
446
447      delete_tree(root->left);       //delete the left branch
448      delete_tree(root->right);      //delete the right branch
449
450      free(root);     //deletes the current node
451
452  }
453
```

7) void char_statistics(...)

- **Description:**
  - Displays the cumulative counts of letters, numbers, dots, and dashes used in encoding and decoding. It saves these statistics in a file named Statistics.txt.
- **Parameters:**
  - int letters: Count of letters processed.
  - int numbers: Count of numbers processed.
  - int dots: Count of dots in Morse code.
  - int dashes: Count of dashes in Morse code.
- **Return type:**
  - None.
- **Notes:**
  - Outputs the statistics to both the console and the Statistics.txt file.
  - If the file cannot be opened, it prints an error message.
  - Provides a summary of the Morse code usage and its character components.
- **Code:**

```
456  //function to calculate character statistics and save to a file
457  void char_statistics(int letters, int numbers, int dots, int dashes) {
458
459      // Print the statistics to the screen so the user can see it
460      printf("\nCharacter Statistics:\n");
461      printf("----------------------\n");
462      printf("-> Letters: %d\n", letters); //shows how many letters there are
463      printf("-> Numbers: %d\n", numbers); //shows how many numbers there are
464      printf("----------------------\n");
465      printf("Morse Characters:\n");
466      printf("-> Dots: %d\n", dots); //shows how many dots there are
467      printf("-> Dashes: %d\n", dashes); //shows how many dashes there are
468      printf("----------------------\n");
469
470
471      //save the statistics to a file called 'Statistics.txt'
472
473      FILE *fp = fopen("Statistics.txt", "a");    //opens the file for writing
474
475      if (fp == NULL) {
476
477          //if the file cannot be opened, print an error message and stop
478          printf("Error opening statistics file for writing.\n");
479          return;
480      }
481
482      //writing the same statistics to the file
483      fprintf(fp, "\nCharacter Statistics:\n");
484      fprintf(fp, "----------------------\n");
485      fprintf(fp, "-> Letters: %d\n", letters);
486      fprintf(fp, "-> Numbers: %d\n", numbers);
487      fprintf(fp, "----------------------\n");
488      fprintf(fp, "Morse Characters:\n");
489      fprintf(fp, "-> Dots: %d\n", dots);
490      fprintf(fp, "-> Dashes: %d\n", dashes);
491      fprintf(fp, "----------------------\n");
492
493      fclose(fp);      //close the file after writing
494      printf("\nStatistics saved to 'Statistics.txt'.\n\n");    //let the user know the statistics were saved
495  }
496
```

8) void reset_statistics(...)

- **Description:**
  - Resets all character statistics (letters, numbers, dots, and dashes) to zero.
- **Parameters:**
  - `int *letters`: Pointer to the count of letters.
  - `int *numbers`: Pointer to the count of numbers.
  - `int *dots`: Pointer to the count of dots.
  - `int *dashes`: Pointer to the count of dashes.
- **Return type:**
  - None.
- **Notes:**
  - Directly modifies the values of the counters by dereferencing the pointers.
  - Used to clear statistics before starting a new encoding or decoding session.
- **Code:**

```
499  //function to reset all the statistics to zero
500  void reset_statistics(int *letters, int *numbers, int *dots, int *dashes) {
501      *letters = 0;
502      *numbers = 0;
503      *dots = 0;
504      *dashes = 0;
505  }
```

9) int main(...)

- **Description:**
- The main function is the entry point of the Morse code program. It initializes the Morse code binary tree, maintains cumulative statistics, supports command-line execution, and provides an interactive menu for encoding, decoding, and viewing statistics.
- **Parameters:**
  - `int argc`: number of command-line arguments.
  - `char *argv[]`: array of command-line argument strings.
- **Return type:**
  - Returns `0` upon successful program execution.

- **Notes:**
  - Initializes the Morse code tree and cumulative statistics.
  - Supports command-line mode (encode/decode + statistics) Performs the operation, print statistics, resets counters, and exits.
  - If run without arguments, the program enters the interactive menu (encode, decode, statistics, exit).
  - Handles invalid menu choices by displaying an error message.
  - Ensures memory allocated for the binary tree and input strings is properly freed before exiting.
- **Code:**

```
59  int main(int argc, char *argv[]){
60
61  //making the root of the Morse code tree. It starts with nothing inside.
62      tree_node *root = create_node('\0');
63      build_tree(root);       //building the whole Morse code tree.
64
65  //variables to keep track of all the letters, numbers, dots, and dashes that was used.
66      int cumulative_letters = 0;
67      int cumulative_numbers = 0;
68      int cumulative_dots = 0;
69      int cumulative_dashes = 0;
70
71
72  // COMMAND LINE SWITCH HANDLING
73  // Case 1: encode from command line: morse.exe 1 hello world
74  if (argc >= 3 && strcmp(argv[1], "1") == 0) {
75
76      // Build the full input text from argv[2] onward
77      char buffer[1024] = "";
78
79      for (int i = 2; i < argc; i++) {
80          strcat(buffer, argv[i]);
81          if (i < argc - 1) {
82              strcat(buffer, " ");   // add space between the words
83          }
84      }
85
86      // Encode the full string
87      encode_morse(buffer, &cumulative_letters, &cumulative_dots,
88                   &cumulative_dashes, &cumulative_numbers);
89
90      // Print statistics AFTER encoding
91      char_statistics(cumulative_letters, cumulative_numbers,
92                      cumulative_dots, cumulative_dashes);
93
94      // Reset statistics
95      reset_statistics(&cumulative_letters, &cumulative_numbers,
96                       &cumulative_dots, &cumulative_dashes);
97
98      delete_tree(root);
99      return 0;
100 }
```

```c
// Case 2: decode from command line: morse.exe 2 .... .-.. ---
if (argc >= 3 && strcmp(argv[1], "2") == 0) {

    // Build the full Morse input from argv[2] onward
    char buffer[1024] = "";

    for (int i = 2; i < argc; i++) {
        strcat(buffer, argv[i]);
        if (i < argc - 1) {
            strcat(buffer, " ");   // keep spaces between Morse groups
        }
    }

    // Decode the full Morse string
    decode_morse(root, buffer, &cumulative_letters, &cumulative_dots,
                 &cumulative_dashes, &cumulative_numbers);

    // Print statistics AFTER decoding
    char_statistics(cumulative_letters, cumulative_numbers,
                    cumulative_dots, cumulative_dashes);

    // Reset statistics
    reset_statistics(&cumulative_letters, &cumulative_numbers,
                     &cumulative_dots, &cumulative_dashes);

    delete_tree(root);
    return 0;
}

// If none of the above matched and argc > 1 → invalid usage
if (argc > 1) {
    printf("Invalid command line usage.\n");
    printf("Usage:\n");
    printf("  %s 1 text        # encode text to morse\n", argv[0]);
    printf("  %s 2 morse_code  # decode morse to text\n", argv[0]);
    delete_tree(root);
    return 0;
}


while(1){   //infinite loop for menu interaction with user

//options for the menu for the user to choose
    printf("Select from the menu:\n");
    printf("1. Encode Morse Code (text to morse)\n");
    printf("2. Decode Morse Code (morse to text)\n");
    printf("3. Character statistics\n");
    printf("4. Exit\n");
    printf("\nMenus item: \n");

    int choice;  //variable to hold the user's choice

    scanf("%d",&choice);
    getchar();  //after reading the user choice, there would be a newline char (\n) left by scanf
```

```
    switch(choice){

    case 1:
        printf ("\nEnter a text: \n");
        char *input_1 = input();    //read input from the user
        encode_morse(input_1, &cumulative_letters, &cumulative_dots, &cumulative_dashes, &cumulative_numbers);  //changing the user's words to Morse code.
        printf ("\n");
        free (input_1);     //free allocated memory
        break;

    case 2:

        printf("\nEnter a Morse code: \n");
        char *input_2 = input();     //read Morse code input from user
        decode_morse(root, input_2, &cumulative_letters, &cumulative_dots, &cumulative_dashes, &cumulative_numbers);  //change the Morse code to regular words.
        printf("\n");
        free(input_2);      //free allocated memory
        break;

    case 3:

        char_statistics(cumulative_letters, cumulative_numbers, cumulative_dots, cumulative_dashes);  //show the user how many letters, numbers, dots, and dashes we've used.
        reset_statistics(&cumulative_letters, &cumulative_numbers, &cumulative_dots, &cumulative_dashes);  // Reset cumulative statistics.
        printf("\nStatistics have been reset.\n");  // Inform user of reset.
        break;

    case 4:

        printf("\nExiting program.\n");
        delete_tree(root);  //free allocated memory for Morse tree

        return 0;       //exits the program

    default:
        printf ("\nInvalid choice. Please try again.\n");  //handling invalid choices

    }

}

    return 0; //ending the program successfully :)
}
```